ZED DIGITAL · PROBLEM STATEMENT 2 · PROJECT REPORT

Gate Counter That Keeps One ID Per Person

BoT-SORT + visibility rules vs the ByteTrack baseline, tested on a seeded gate simulator and one labelled real clip

Author	Institution	Date	Code
Panshul Arora	SRM IST	17 October 2026	github.com/panshularora/zed-ps2-tracker · branch `feat/stress-map`, head `2998654` · draft PRs #43 and #44

1. Summary

At a ticket gate, a people counter is only as good as its IDs. If a person is hidden behind someone for a second and comes back with a new number, the gate can miss their crossing, count them twice, or hand their ticket to someone else. This project builds a strictly online tracker that tries to keep **one ID per person from approach to exit**, a gate-counting layer on top of it, and a web app that shows, honestly, where it works and where it fails.

Our tracker is **BoT-SORT plus a visibility rule**: when a person's last box looked mostly hidden (estimated visibility below 0.7), the tracker remembers them three times longer (90 frames, 3 s at 30 fps, instead of 30). The baseline is **ByteTrack** with its code defaults. Both run from the same from-scratch `tracker/` code.

Key results (every number is from the repo files listed in the appendix)

- **Stress Map** (synthetic gate, 48 cells × 20 seeds): ours has fewer ID changes than ByteTrack in **24 of 48** cells and more in **14 of 48** (tie 4, unclear 6). Ours wins at every crowd size for hides of 1–2.5 s; past 3 s its memory runs out.
- **Longer memory** (opt-in `visibility_botsort_mem6.yaml`): better 36, **worse 1**, tie 1, unclear 10. Tuned on separate seeds; on real data it was checked on MOT17 only (HOTA 52.5 either way). Not the default.
- **MOT17-09 upload** (the only real clip with ground truth, CPU, YOLOX-S): truth **3 IN / 10 OUT**, tracker counted **4 IN / 11 OUT**. **9 of 15** counted crossings are real and **4** true crossings were missed.
- **Official benchmarks** (GPU detectors): **the visibility rule stops re-ID from hurting**: MOT20 HOTA ByteTrack 69.4 → + re-ID 67.5 → ours 69.5; MOT17 62.5 → 63.2. Not the best tracker overall (BoT-SORT wins MOT17, Hybrid-SORT DanceTrack). ByteTrack MOTA 66.4 vs 76.6 published, so every GPU number is provisional.
- **Engineering**: 335 pytest and 187 vitest tests; ruff, mypy, tsc and eslint clean. The CI workflow file still needs a `.yml` extension before GitHub runs it.

Status. All work is in two draft pull requests: #43 (review fixes, upload in Studio, stress suite) and #44 (Stress Map, cell replay, Break it yourself, raw-frame tracking), stacked on `main` at `7559df8`. Nothing has been merged.

2. The problem

Zed's ZIG platform uses crowded gates for hands-free validation, Fast Lane entry, tailgating control and passenger counting. Each of these needs the tracker to keep one body attached to one number (from the repo README):

Gate use	What it needs from the tracker	What breaks it
Hands-free validation, Fast Lane entry	A validated ticket stays on the right body through the gate	An ID switch hands one person's validation to someone else
Tailgating / fare-evasion control	Two bodies on one validation are two distinct IDs	An ID merge hides a tailgater; a split invents a phantom
Passenger counting	One crossing per person per direction	Fragmented tracks double-count; merged tracks under-count
Privacy	Identity without identification	Stored faces or long-lived identities

We focus on three failure modes:

- **ID switch after a hide.** A person walks behind someone, the detector loses the box, the tracker's memory expires, and the person comes back with a new number. Example trace from the stress suite (10 people, 1 s hidden, seed 1000, ByteTrack): "#3 was deleted at frame 154 after 31 frames without a box (memory 30); the person's next box started #9."
- **Swap.** The same number comes back, but on a different person. On the subway-turnstile demo clip, #14 is last seen at frame 230 on a woman in a black top and drawn again at frame 318 on a man in a white shirt (`docs/keep_check.md`). A naive "kept #14" label would be wrong.
- **Missed crossing.** Counting needs the same ID on both sides of the line. If the ID changes while the person is hidden on the line, neither the old nor the new number crosses, so the crossing is lost. In the stress grid the gate error is negative for both trackers in every cell, and at hides of 3 s or more it is about −(number of people).

3. Our approach

Baseline: ByteTrack (`configs/baselines/bytetrack.yaml`). Two-stage association on IoU: high-score boxes (≥ 0.6) first, then low-score boxes (≥ 0.1); new tracks need 0.7; match threshold 0.9; lost tracks are kept for **30 frames**; no appearance features.

Ours: BoT-SORT + visibility (`configs/ours/visibility_botsort.yaml`). BoT-SORT's tracker part (ByteTrack association, xywh Kalman filter, camera-motion compensation when a CMC file exists; appearance re-ID **off** by default) plus a depth-order visibility estimate: each box's overlap with closer boxes, multiplied by the detector score. The rule changes only one thing, **how long a lost track is remembered**: if its last box had visibility below 0.7, the 30-frame buffer is multiplied by 3 (90 frames). Display code never relabels IDs; an earlier "Keep-ID" relabelling layer was found in review and deleted.

Config	Builds on	Key settings	Status
`baselines/bytetrack.yaml`	—	high 0.6 / low 0.1 / new 0.7; match 0.9; buffer 30; no appearance	Baseline everywhere
`ours/visibility_botsort.yaml`	`baselines/botsort.yaml`	depth-order visibility × score; `occluded_thresh` 0.7; `occluded_buffer_scale` 3.0 (30 → 90 frames)	**Default "ours"** in app, API and stress suite
`ours/visibility_botsort_mem6.yaml`	`visibility_botsort.yaml`	`occluded_buffer_scale` 6.0 (180 frames, 6 s at 30 fps)	Opt-in only

Config	Builds on	Key settings	Status
`ours/visibility.yaml`	ByteTrack	same visibility rule on ByteTrack	Measured, superseded by the BoT-SORT row
`ours/delayed*.yaml` (v1, v2_reid, v3_strict)	ByteTrack (+ re-ID)	K-frame look-ahead on ambiguous matches	**NO-GO**; off by default

What we tried and dropped. Delayed assignment (delay-K, pre-registered rule) failed all five checks on the GPU run: IDSW 183 → 198 at K = 5 instead of −10 %, IDF1 73.0 → 71.9, it fired on about 51 % of frames vs a 10 % limit, p95 170 ms vs 20 ms, and MOT20 IDSW 1462 → 1575; so it was dropped (NO-GO). Buffered-IoU variants beat the longer memory on the synthetic tuning set but cost 0.7–1.5 IDF1 on MOT17, so they were removed. On MOT17 with public detections, the visibility rule is about as good as simply using a longer buffer for every track (table below).

MOT17 val-half, public FRCNN, CPU, no re-ID	HOTA	IDF1	IDSW
ByteTrackLike (baseline)	50.1	56.3	191
BoT-SORT (CMC)	51.6	58.8	197
BoT-SORT, buffer 90 for every track (control)	52.4	59.5	180
BoT-SORT + visibility (ours, default)	**52.5**	**59.8**	180
BoT-SORT + visibility, longer memory (mem6)	52.5	59.7	181

These CPU numbers use public detections and IoU-only association. They are a pipeline and guard-rail check, not comparable to published YOLOX results.

4. Architecture and data flow

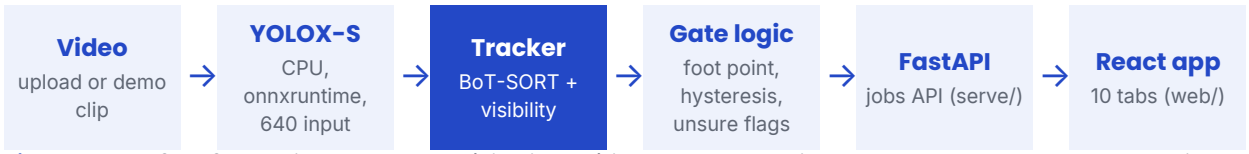


Figure 1. Data flow for one job. The tracker (highlighted) is the same code in the app, the API and the stress suite.

Stage	What happens	Where
Video in	`POST /api/uploads` (mp4/mov/m4v/webm/mkv/avi, with a consent flag). Limits: 200 MB, 120 s, 1,800 tracked frames per job, 1 job worker.	`serve/api.py`, `serve/settings.py`
Gate	The user clicks two points on a face-blurred first frame (`GET /api/clips/{id}/first-frame.jpg`); the job carries the line.	Studio step 4
Detection	YOLOX-S (default `ZED_DETECTOR`), official ONNX release 0.1.1rc0, COCO weights, 4 onnxruntime threads.	`serve/detector.py`
Tracking	`ours/visibility_botsort` on the raw frames, decoded in memory. Re-ID off on CPU (`tracker_info.reid = false`).	`tracker/`
Gate logic	Crossing = box bottom-centre crosses the line with hysteresis. Crossings where visibility or score was under 0.5 in the half-second before the line are flagged unsure. A separate step lists possible missed crossings (an ID ending at the line paired with a new one starting across it); these are never added to the count.	`eval/gate_events.py`, `eval/crossing_recovery.py`, `serve/analysis.py`
Outputs	`GET /api/jobs/{id}` status and result, `tracks.csv`, `annotated.mp4`, `events.json`; cancel and delete.	`serve/api.py`
Web app	Ten tabs: Dashboard, Overview, What we use, Gate scene, Scenarios, Studio, Results, Experiments, Analytics, Architecture. Falls back to a committed offline snapshot when the backend is down.	`web/src/App.tsx`

Privacy by design

- **Tracking on raw frames, in memory only.** Detection and tracking read the unblurred frames decoded in memory; they are never written to disk. So the blur can never change a count (tested: tracker output is identical with face-oval, head-square or no blur).
- **Oval face blur on output only.** Everything shown or saved (served video, first-frame still, poster, annotated render) uses the face-only soft-ellipse blur, about 1.2× each face, bodies left sharp. Faces come from two CPU detectors so one covers the other's misses: OpenCV YuNet and a face-sized box at the top of every YOLOX person box. The code says this is best effort, not a guarantee.
- **Raw upload deleted after the job.** The original file is deleted when its first job ends (succeeded, failed or cancelled); an upload that never gets a job is swept after `ZED_UPLOAD_RAW_TTL_S` (default 3,600 s). After that the API returns `410 original_gone`. What stays: `blurred.mp4`, `first_frame.jpg`, `meta.json` and job outputs.

5. The app: a short tour

The web app has ten tabs. The screens below are the ones a reviewer needs; every number on them is read from the repo's own output files. Captions marked “earlier capture” are from a previous screenshot pass of the Stress Map, cell replay and Break it screens; they still show those screens accurately.

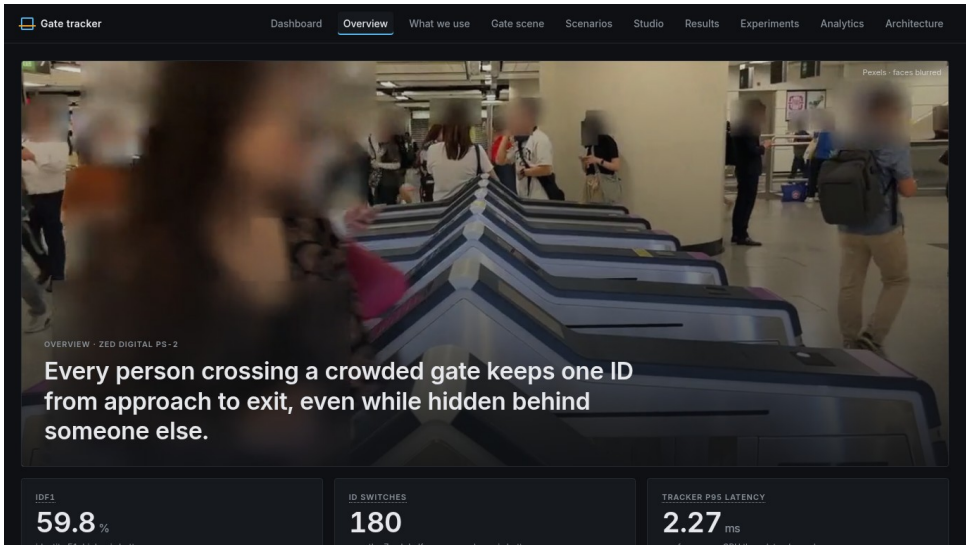
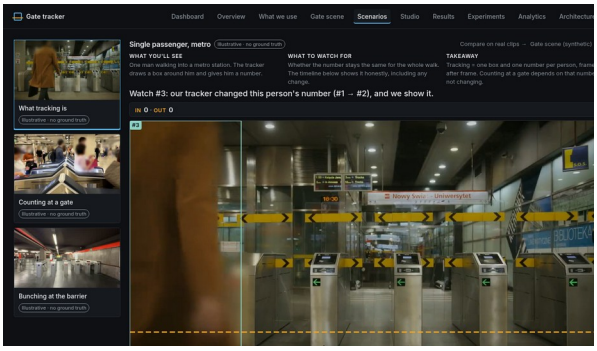


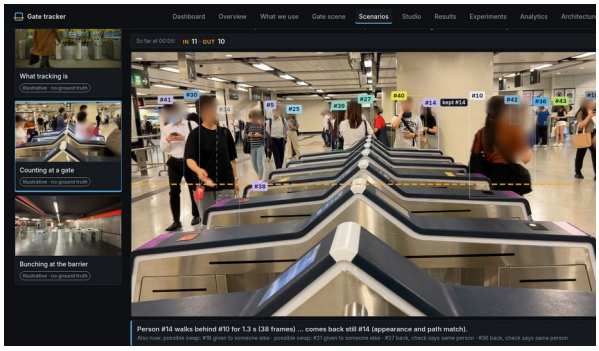
Figure 2. Overview tab. The project's one-line goal over a face-blurred gate clip; the cards below show the CPU MOT17 numbers for ours (IDF1 59.8, 180 ID switches, tracker p95 latency 2.27 ms).

Scenarios

Short real clips (Pexels, no ground truth, so each is labelled “Illustrative · no ground truth”). Each scenario says what you will see, what to watch for and the takeaway, then plays our tracker exactly as it tracked, including its failures. Only the focus person's event appears on the frame (for example `kept #14`); other events go to the caption bar and the timeline.

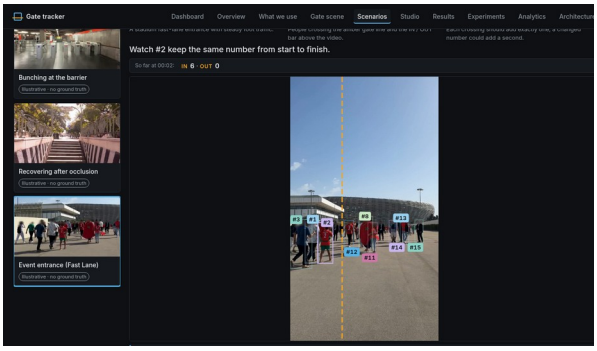


What tracking is

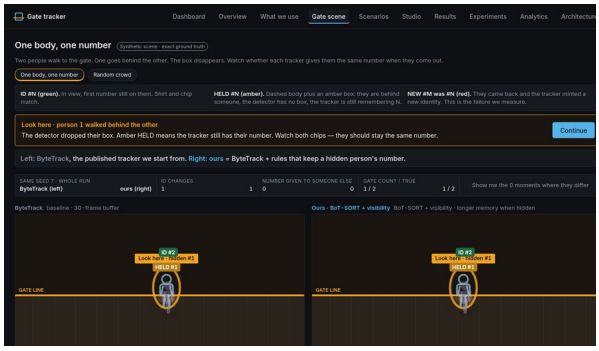


Counting at a gate: “kept #14” tag

Figure 3. Left: the first scenario openly shows our tracker changing a person's number (#1 → #2). Right: subway turnstiles, where #14 walks behind #10 for 1.3 s (38 frames) and comes back still #14; the caption bar also lists two possible swaps.



Event entrance (Fast Lane)



Gate scene: one body, one number

Figure 4. Left: the portrait Fast Lane clip is letterboxed, not cropped, so boxes and the gate stay aligned. Right: Gate scene, a synthetic two-person script with exact ground truth comparing ByteTrack (left) and ours (right).

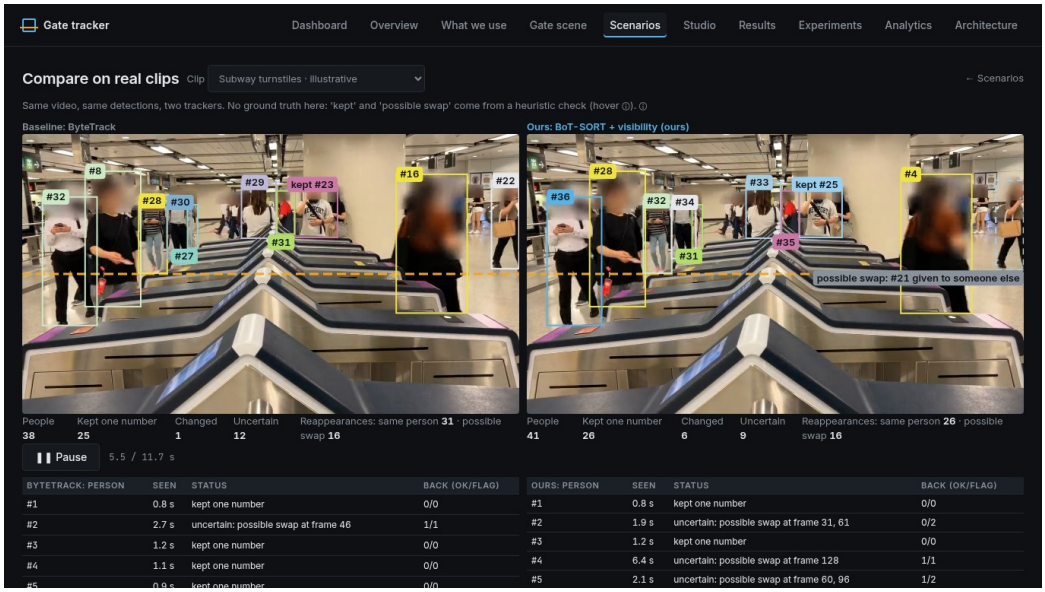


Figure 5. Compare on real clips: same video, same detections, two trackers. “Kept” and “possible swap” come from a heuristic check (colour, path, size), because these clips have no labels.

Studio, including step 4 “Your video”

Studio is a four-step workstation: **1 Watch the failure, 2 Compare the trackers, 3 See the evidence, 4 Your video**. Step 4 is the real upload path: choose a file, tick consent, click two points for the gate on the blurred first frame, run, and open the result in the Studio player.

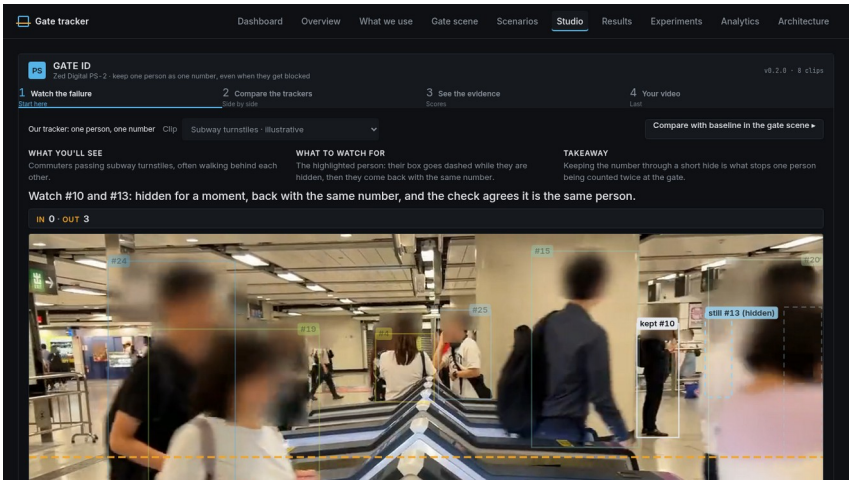
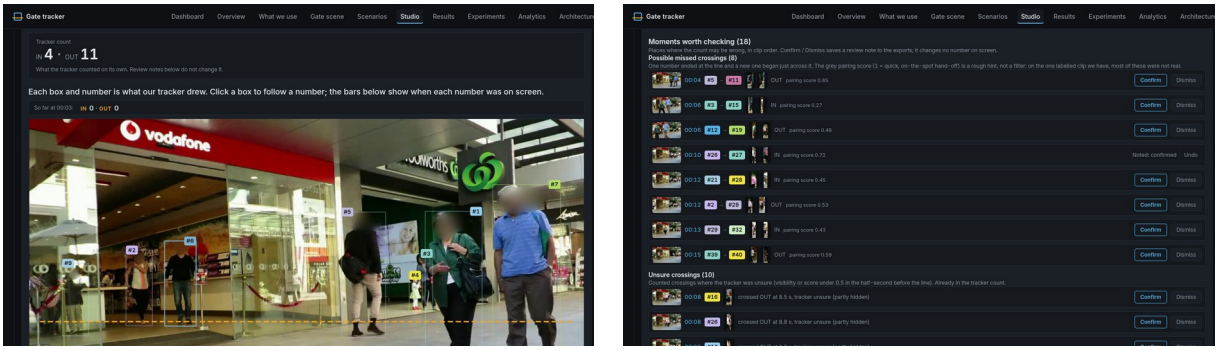


Figure 6. Studio step 1 on the subway clip: #10 and #13 are hidden for a moment; the dashed box marks a hidden person still being remembered.



Tracker count and player — Moments worth checking

Figure 7. Step 4 on the MOT17-09 upload. Left: one "Tracker count" card (4 IN / 11 OUT) and the player with a live "So far" strip. Right: "Moments worth checking (18)": 8 possible missed crossings and 10 unsure crossings, in clip order, each with a still, ID chips and Confirm / Dismiss. These are review notes saved to the exports; they never change the count.

Results: Stress Map, cell replay, Break it yourself

The **Stress Map** opens the Results tab: an 8 × 6 grid (hidden time × people in view). Each cell is the mean of ours minus ByteTrack in ID changes over 20 seeded runs: blue means ours made fewer ID changes, amber more, hatched not clear. A toggle switches to the opt-in longer memory. Borders and the headline are computed from the data.

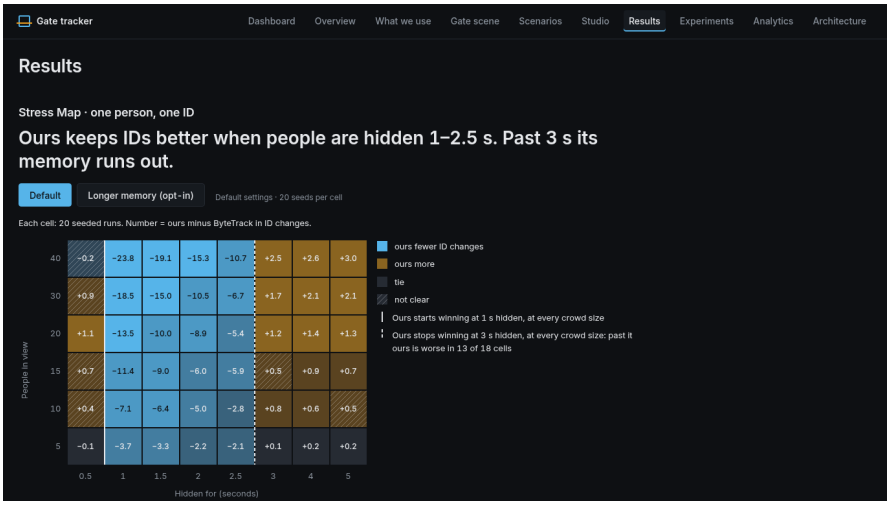
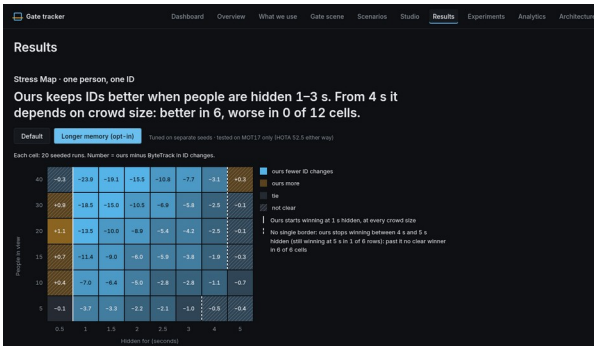
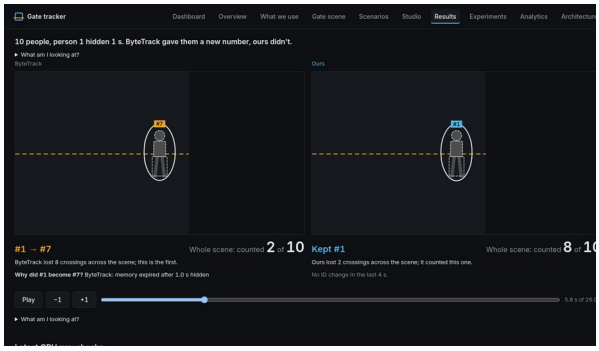


Figure 8. Stress Map, default config (earlier capture). Ours wins from 1 s hidden at every crowd size and stops winning at 3 s. Cell values are rounded to one decimal, so a few may differ by 0.1 from the tables in section 7.

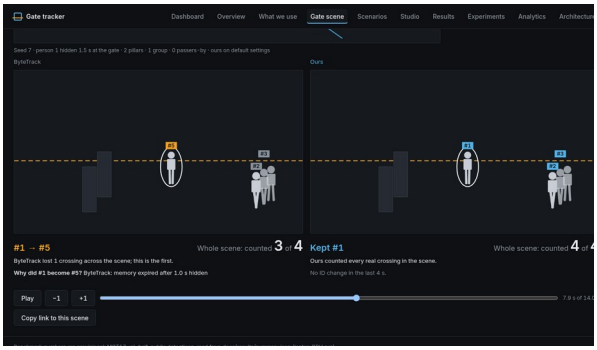


Longer memory toggle

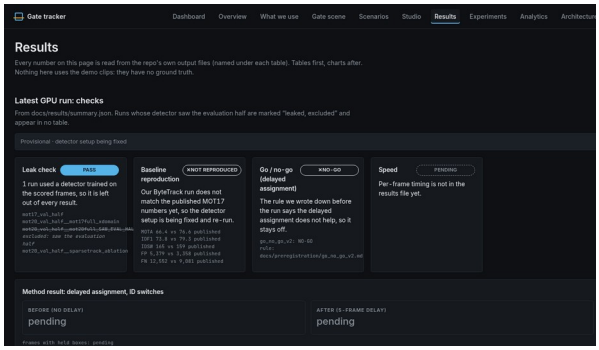


Cell replay with the reason

Figure 9. Left: the same map with "Longer memory (opt-in)"; the winning band extends to 3–4 s. Right: clicking a cell replays its median seed side by side. Here (10 people, 1 s hidden) ByteTrack turns #1 into #7 because its "memory expired after 1.0 s hidden"; ours keeps #1. Whole scene: counted 2 of 10 vs 8 of 10. (Earlier captures.)



Break it yourself + shareable link



Results: GPU-run checks

Figure 10. Left: Break it yourself (Gate scene). Drag pillars or groups, draw a path, hide a person; both real trackers run on the deterministic simulator, and "Copy link to this scene" gives a shareable URL that rebuilds the same scene. Right: the GPU-run checks on Results, showing the baseline as NOT REPRODUCED and the method result filled from the 18 Oct GPU run (provisional).

6. Evaluation method

Real gate clips with labels are rare (MOT17-02/04/09 contain only 25 crossings), so the main evidence is a **seeded gate simulator** where the truth is exact. It tests the association logic, not real-video accuracy.

Item	Setting
Scene	960 × 540 at 30 fps, counting line at y = 300. Person-shaped boxes walk through; each scripted person is fully hidden for the chosen time (plus a 6-frame fade in and out), centred on the line.
Detector noise	Seeded box jitter, missed boxes, false boxes, score noise, truncation of partly hidden boxes. Noise level 0.15 in the grid; a sweep tests 0.05, 0.15 and 0.4.
Grid	8 hidden times (0.5, 1, 1.5, 2, 2.5, 3, 4, 5 s) × 6 crowd sizes (5, 10, 15, 20, 30, 40 people) = **48 cells**; plus 10 scenario kinds (look-alike crossing, stop-and-go, step-back, group, camera shake, linger, fast walker, 5/15/30-frame dropouts).
Seeds	**20 per cell, seeds 1000–1019**, all reported, none dropped (960 runs per tracker on the grid). Tuning used **separate seeds 0–9 only**.
Metrics	ID changes per clip; strict kept (one ID covers ≥ 80 % of the person's visible frames and is not used on anyone else); gate count error; IDF1, AssA, HOTA, IDSW, Frag from the vendored **TrackEval**.
Verdict per cell	Per-seed difference ours − ByteTrack in ID changes. **Better** if the bootstrap 95 % CI (4,000 resamples) is below 0 **and** the exact sign test gives $p < 0.05$; **worse** if the CI is above 0 and $p < 0.05$; **tie** if all differences are 0 or the CI is within ±0.5; else **unclear**.
Explainability	Where the two trackers' ID histories diverge, the tracker's own trace records why (memory expired, cost above threshold, box taken by another track, and so on).

Item	Setting
Real-video check	MOT17-09 uploaded through the app. Ground truth (`gt.txt`) is checked offline only, on the same line, matching crossings one-to-one in the same direction within 15 frames. Never shown in the UI.

Reproduce: `python -m eval.stress_run grid --seeds 20 --seed0 1000 --workers 6` (add `--ours configs/ours/visibility_botsort_mem6.yaml --tag _mem6` for longer memory). Runtime on the box: 268 s (default) and 404 s (mem6) with 6 workers.

7. Results

7.1 Stress Map: 24 better, 14 worse

Verdict per cell, ours (default) vs ByteTrack. B = ours better, W = ours worse, T = tie, U = unclear. Numbers are mean ID changes per clip, ours / ByteTrack.

	0.5 s	1 s	1.5 s	2 s	2.5 s	3 s	4 s	5 s
5 people	1.0/1.1 **T**	1.4/5.0 **B**	1.7/5.0 **B**	2.8/5.0 **B**	3.0/5.0 **B**	5.1/5.0 **T**	5.2/5.0 **T**	5.1/4.9 **T**
10 people	2.8/2.5 **U**	3.5/10.5 **B**	4.2/10.6 **B**	5.3/10.3 **B**	7.6/10.3 **B**	10.9/10.2 **W**	10.7/10.1 **W**	10.4/10.0 **U**
15 people	4.5/3.9 **U**	5.0/16.4 **B**	7.1/16.1 **B**	9.8/15.8 **B**	10.0/15.8 **B**	16.2/15.8 **U**	16.1/15.2 **W**	15.3/14.7 **W**
20 people	8.4/7.3 **W**	9.1/22.5 **B**	12.4/22.4 **B**	12.6/21.4 **B**	16.6/21.9 **B**	22.7/21.6 **W**	21.9/20.5 **W**	20.7/19.4 **W**
30 people	15.9/15.0 **U**	16.6/35.0 **B**	20.2/35.1 **B**	24.0/34.5 **B**	26.8/33.5 **B**	35.3/33.6 **W**	33.9/31.8 **W**	31.6/29.6 **W**
40 people	23.8/24.0 **U**	27.4/51.1 **B**	31.6/50.6 **B**	33.6/48.9 **B**	37.5/48.1 **B**	49.3/46.9 **W**	46.8/44.1 **W**	43.4/40.5 **W**

- **Ours is clearly better for hides of 1–2.5 s at every crowd size** (all 24 “better” cells). Example, 10 people hidden 1 s: 3.5 vs 10.5 ID changes per clip, strict kept 6.6 vs 0.0 of 10, IDF1 range 0.71–0.94 vs 0.49–0.56.
- **Ours is worse in 14 cells**: 20 people at 0.5 s (8.4 vs 7.3, CI [+0.1, +2.0], p = 0.022) and 13 of the 18 cells at 3 s or more, where both trackers give nearly every hidden person a new number.
- The two trackers' ID histories diverge in **951 of 960** seeds. Most common reason at the first divergence: ByteTrack's 30-frame memory expired (558 seeds).
- **Scenarios:** ours is better in 7 of 10 kinds (for example step-back 3.60 vs 10.45 and fast walker 3.45 vs 10.65 ID changes). In the three abrupt-dropout kinds it is not better (+0.3 / +0.4 / +0.4, unclear), because the rule only extends memory when the last box looked occluded.

7.2 Longer memory: 1 worse cell, opt-in, real-data check on MOT17 only

Verdicts over 48 cells	Better	Worse	Tie	Unclear	Sum of mean ID changes
Ours default (`visibility_botsort`)	24	14	4	6	786.7
Ours longer memory (`…_mem6`, opt-in)	36	**1**	1	10	726.1

Longer memory turns 13 of the 14 “worse” cells into better or unclear and changes cells below 3 s by at most 0.2 ID changes per clip. The one remaining worse cell is 20 people at 0.5 s. It was tuned on seeds 0–9 and its only real-data check is MOT17 val-half on CPU (HOTA 52.5 / IDF1 59.7 / IDSW 181 vs 52.5 / 59.8 / 180), so it stays **opt-in** and the default is unchanged. Strict kept at 3 s or more is still low (for example 5.4 of 40 people at 3 s).

7.3 The MOT17-09 upload (real video, ground truth checked offline)

MOT17-09 (960 × 540, 525 frames, 30 fps) went through Studio step 4 like any user video: gate clicked at (0.02, 0.65) → (0.98, 0.65), ours on CPU with YOLOX-S, canonical job `03bcd2ab3ace`, tracked on raw frames.

Measure	Value
True count (from `gt.txt`, offline)	**3 IN / 10 OUT**
Tracker count shown in the app	**4 IN / 11 OUT**
Counted crossings that match a true one	**9 of 15** (precision 0.60)
True crossings the tracker missed	**4**
Possible missed crossings listed	8, of which 1 is real (2nd by pairing score; the top-scored one is not real)
Unsure crossings listed	10
Recorded experiment: add the 1 real suggestion	IN 4 → 5, away from the true 3 (6 false counted crossings stay in; suggestions can only add)

So the net count is close (off by +1 / +1), but that hides errors on both sides: 6 false crossings and 4 missed ones. This is why the app shows only the tracker count, and lists suggestions as **moments worth checking**, never as an adjusted count. Crossing recovery also made the count worse on the frozen MOT17-09 gates for both trackers.

7.4 Demo clips' tallies (no ground truth)

The six Pexels demo clips were re-cut with the face-oval blur and re-tracked on raw frames. Tallies from PR #44, ByteTrack / ours, old → new:

Clip	ID switches	Tracks	Disagreements
man_entering_metro	0/0 → 0/0	1/3 → 1/1	0 → 0
subway_turnstile	8/13 → 15/16	45/56 → 58/74	25 → 35
milan_turnstiles	0/0 → 1/0	3/4 → 10/11	0 → 0
mexico_city_entrance	0/0 → 0/0	7/8 → 10/11	0 → 0
shibuya_crossing	20/28 → 30/35	104/130 → 125/155	81 → 72
rabat_stadium	2/4 → 5/5	22/30 → 21/24	5 → 1

Reappearance checks (kept / possible swap / unknown, `docs/keep_check.md`), ours vs ByteTrack: subway_turnstile 41/29/0 vs 42/35/0; shibuya_crossing 50/67/0 vs 48/75/0; MOT17-09 with ground truth 11/2/0 vs 11/2/1; MOT17-04 with ground truth 11/1/0 vs 11/4/0. On Pexels clips this is a colour/path/size heuristic: on 53 labelled MOT17 reappearances the path and size checks passed all 9 real swaps, so the colour check does most of the work and has not been validated against labels. On most of these clips ours has as many or more ID switches than ByteTrack (subway 16 vs 15, Shibuya 35 vs 30); there is no ground truth to say which is right.

7.5 Official benchmarks: provisional GPU run

Dataset (GPU detector)	IDF1 ByteTrack	IDF1 ours	AssA ByteTrack	AssA ours	Note
MOT17 val-half	73.8	75.1	66.5	67.1	Provisional; MOTA 66.4 vs 76.6 paper
MOT20 val-half	84.1	84.0	66.8	66.3	SparseTrack cache, leak-free; transfer also ran
DanceTrack val	54.9	55.8	36.3	38.8	Complete; vis-gating idle (no vis 0-0.25 boxes)

Scored with vendored TrackEval on validation-half (MOTChallenge test server is offline).

Main finding: the visibility rule stops re-ID from hurting. MOT20 val-half with the SparseTrack detector: ByteTrack 69.4 HOTA → ByteTrack + re-ID 67.5 → ours (re-ID + visibility-gated appearance) 69.5, the best HOTA in that setting (IDSW 1,011 → 1,278 → 1,019). MOT17 val-half: ByteTrack + re-ID 62.5 → ours 63.2. It is not the best tracker overall: BoT-SORT without re-ID beats ours on MOT17 (HOTA 64.6 vs 63.9), and Hybrid-SORT wins DanceTrack val (57.4 vs 54.3) and MOT20 with the MOT17-trained detector (62.9). The table above lists ours = BoT-SORT + re-ID + visibility; the ByteTrack-based ours reaches IDF1 84.5 on MOT20.

8. Limitations and where we lose

- **Memory past 3 s.** The rule keeps an occluded track for 90 frames (3 s). Past that, ours expires too and is slightly worse than ByteTrack in 13 of 18 cells; strict kept is about 0 and the gate misses about every hidden person. Longer memory helps but is opt-in and was checked on real data only on MOT17.
- **Short hides in big crowds.** At 0.5 s with 20 people ours is worse (8.4 vs 7.3), with or without longer memory. Abrupt dropouts get no benefit.
- **ByteTrack GPU baseline not reproduced.** On the laptop GPU run (RTX 3050, YOLOX-X), our ByteTrack gave MOTA 66.4 vs 76.6 published, IDF1 73.8 vs 79.3, IDSW 165 vs 159, FP 5,379 vs 3,358, FN 12,552 vs 9,081. False positives and misses are far worse while ID switches are similar, which points at the detections, not the tracker. MOT17, MOT20 and DanceTrack val ablations finished on this cache; numbers stay provisional until the paper MOTA gap closes. **The root cause is not confirmed**; the 18 Oct GPU rerun finished on this cache.
- **Other trackers win some datasets.** DanceTrack val: Hybrid-SORT 57.4 HOTA vs ours 54.3. MOT20 with the MOT17-trained detector: Hybrid-SORT 62.9 vs ours 61.1. MOT17: BoT-SORT without re-ID 64.6 vs ours 63.9.
- **Speed with re-ID.** On MOT20 (SparseTrack cache) ours needs 87–95 ms per frame at p95 vs 13.7 ms for ByteTrack (IoU), on the laptop GPU.
- **No re-ID on CPU.** Appearance features are off in the app and the stress suite, so ours relies on motion and memory only. The swap check on Pexels clips uses colour histograms instead.
- **Synthetic evidence.** The simulator moves boxes, not pixels, and its detector is a noise model. One shipped synthetic crowd recording (seed 7, 30 s) is a counter-example: ID changes ours 25, BoT-SORT 26, ByteTrack 21; all counted 25 of 34.
- **Upload counts are not accurate.** Beyond MOT17-09: on Shibuya a horizontal line counted 0 / 0 because foot points never reach it (a vertical line counted 0 / 4); the gate editor gives no hint where foot points are. Yonge–Dundas counted 1 / 1 with 87 IDs, a likely under-count with no truth to check.
- **"Make it harder" is not monotonic.** A tracker can keep a number at a longer hide after losing it at a shorter one, so the search cannot stop at the first break. It is hidden behind the `harder` feature flag, off by default.

Superseded runs and why they changed

Run	Result	Why it changed
MOT17-09 upload, head-square blur (job `367de74b7ea3`)	0 IN / 7 OUT; 9 suggestions, 3 real	Detection read the **blurred** copy, so the blur changed the counts. The UPLOAD_E2E report's 0/7 is this setup.
MOT17-09 upload, face-oval blur (job `d90a457d1cab`)	3 IN / 9 OUT; 6 suggestions, 1 real	Still tracked on the blurred copy.

Run	Result	Why it changed
MOT17-09 upload, canonical (job `03bcd2ab3ace`)	**4 IN / 11 OUT**; 8 suggestions, 1 real	Detection and tracking on raw frames; blur on output only (commit `89ba09f`). A later screenshot re-run (`881ffbdd57d3`) was byte-identical.
"Operator-adjusted" count card	Removed	Confirming the real suggestion moved IN 4 → 5, away from the true 3.
Demo-clip tallies and keep checks	Regenerated (table 7.4)	Clips re-cut with the oval blur and re-tracked on raw frames; old story moments that turned out to be different people were dropped.
Keep-ID relabelling on `main`	Deleted	It rewrote ours' new numbers back to old ones, so the "Ours" pane was not our tracker.

9. Engineering quality

Check	Result on feat/stress-map
pytest (backend, tracker, eval, serve)	**321 passed** (re-run on head `2998654`, 79 s on the box CPU)
vitest (web app)	**175 passed** in 24 test files (re-run on head `2998654`)
ruff check, ruff format	clean (per PR #44)
mypy (`--strict` on `tracker/`; `eval models serve`)	clean (per PR #44)
tsc --noEmit, eslint, vite build	clean (per PR #44)
CI	`.github/workflows/ci` has no `.yml` extension, so GitHub Actions ignores it. Same file in `docs/ci/github-actions.yml`. **Needs a rename to `ci.yml`** by the repo owner.

For context, `main` at `7559df8` failed `make check` at review time (7 ruff errors, 3 files needing reformat, 1 mypy error); the branch fixes all of them. The tracker depends only on NumPy, SciPy and PyYAML, is a from-scratch re-implementation, and avoids AGPL code. Project code is Apache-2.0; ported tracker code and TrackEval are MIT. The datasets and public weights are non-commercial research use only, and the demo clips are under the Pexels License.

10. Next steps

1. **Close the ByteTrack paper gap** (MOTA 66.4 vs 76.6) or stop quoting paper-comparable numbers; MOT17, MOT20 and DanceTrack val already ran (`docs/results/GPU_RUN_2026-10.md` records the reproduction audit).
2. **Test appearance re-ID on GPU**, gated by visibility, to cover hides past 3 s and abrupt dropouts that motion alone cannot fix.
3. **Decide on longer memory** with more real labelled data than MOT17 before making it the default.
4. **Gate editor**: show foot-point trails on the first frame, or warn when no track crosses the line.
5. **Small fixes from the review**: rename the CI file; let "Your video" resume a running job after a reload; label the two different track counts differently; delete the unused `Demo.tsx`; remove the leftover temp file when a blur is cancelled; wire the two benchmark slots in Scenarios to real exports.
6. **Rework the "Make it harder" scene** so the result is easy to read, then consider turning the flag on.

11. Appendix

How to run

`make setup` Python 3.11 virtualenv with the package and dev tools

`make models clips blur` ONNX models (sha256-checked), Pexels clips, face-blurred copies

`make demo` API on :8000 and web app on http://localhost:5173 (or `docker compose up --build`)

`make check && make test` ruff + mypy + pytest, then pytest + vitest

`make data && make smoke` MOT17/MOT20 labels and public detections; TrackEval + stress + latency

`python -m eval.stress_run grid --seeds 20 --seed0 1000 --workers 6` Stress Map grid (also `scenarios`, `noise`)

`python -m eval.operator_demo` MOT17-09 upload check against ground truth (offline)

Key files

File	What it holds
`docs/review/FULL_REVIEW_2026-10-17.md`	Full code review: P0/P1/P2 findings and fixes
`docs/review/UPLOAD_E2E_2026-10-17.md`	Upload path end to end on five clips
`docs/results/stress/README.md`, `stress_map.json`, `stress_map_mem6.json`	Stress suite method, all grid tables, raw per-seed data
`docs/results/stress/CROSSING_RECOVERY.md`, `operator_demo_mot17_09.json`	Missed-crossing suggestions and the MOT17-09 upload check
`docs/results/GPU_RUN_2026-10.md`	Laptop GPU run, why it is provisional, rerun commands
`docs/keep_check.md`	Kept-vs-swap check on reappearances
`configs/ours/*.yaml`, `configs/baselines/*.yaml`	Tracker configs
`tracker/`, `eval/`	Tracker code; evaluation, simulator, gate events
`serve/api.py`, `serve/settings.py`, `serve/blur.py`	Jobs API, limits and env vars, face blur
`web/src/App.tsx`, `web/src/stress/`	Web app tabs; Stress Map, replay, Break it yourself

Unknowns: tracker latency on the GPU detector path and any production-site accuracy have not been measured. No team name is given in the source files, so none is listed.